

CogniFlow - Demo Video Recording Guide

Written for the second half of Team BloodCoded. Everything you need to record the 3-5 minute demo video for the Agentic AI Hackathon, Tech Zephyr 4.0.

Read this first

You do not need to understand every line of the code to do this well. You need to understand **one idea** deeply, be able to say it in your own words, and be able to point at the place on screen where it happens. That is the whole job. Sections 1 to 3 teach you the idea. Section 6 is the run sheet you actually read while recording.

How to use this document

Section	What it is	When to read it
1. The one idea	What CogniFlow does, in plain English	First. Twice.
2. The five concepts	Every term you might be asked about	Over 2-3 sittings
3. Reading the screen	What each line of output means	With the app open
4. Your job	Which parts are yours to say	Before rehearsing
5. Setup	Exact commands, in order	Recording day
6. Run sheet	Timed script, what to click and say	While recording
7. If it breaks	What to do live	Skim, then trust it
8. Questions	What judges may ask you	Before the finale
9. Glossary	Every term in one place	Whenever stuck
10. Learning plan	What to study before 9 September	Today

1. The one idea

If you remember nothing else, remember this paragraph. It is the entire project.

The situation

A student is learning Python. They try a recursion exercise and get it wrong. They try again and get it wrong again.

What every other AI tutor does

It gives them an **easier recursion question**. Maybe it adds a hint. Maybe it explains recursion again, more slowly.

Why that fails

Very often the student's real problem is not recursion at all. It is that they do not understand what return does when one function calls another. That is a **functions** problem. Recursion is just where it became visible. So the easier recursion question fails too, for exactly the same reason, and the student concludes they are 'bad at recursion' when nobody ever taught them the thing recursion is built on.

What CogniFlow does instead

It notices the pattern, works out that the real gap is **functions**, **changes its own goal** to teach functions, teaches that, and then comes back to recursion - which it never forgot was the original target.

recursion	->	functions	->	recursion
(failing)		(the real		(back to what
		gap)		they wanted)

The sentence to memorise

"Most AI tutors personalise **which question** you get. CogniFlow personalises **the route through the knowledge**."

Everything else in this guide is detail underneath that sentence.

2. The five concepts

These are the five things a judge might ask about. Each one has: what it is, why it matters, and the one sentence you should say. You do not need the mathematics.

2.1 The prerequisite graph (the core)

What it is. A map of which topic depends on which. You cannot understand recursion without understanding functions first. You cannot understand functions without understanding variables. We wrote that map down as a diagram the program can walk through.

```
variables -> functions -> recursion -> recursion_tree
variables -> loops      -> nested_loops
```

Why it matters. Because it lets the program ask a question no chatbot can ask: 'this student keeps failing recursion - what does recursion *depend on*, and are they weak at that instead?'

Say:

"It walks a prerequisite graph. Recursion depends on functions, so when recursion keeps failing it checks whether functions is the real problem."

2.2 Mastery, and why it is not a score out of ten

What it is. For every skill we keep a number between 0 and 1 - our best estimate of the chance the student actually knows it. Recursion at 0.35 means "we think there is a 35% chance they have got it".

The technique is called Bayesian Knowledge Tracing, from a 1995 paper by Corbett and Anderson. You do not need the equations. You need to know why it beats just counting right answers:

- It knows people **guess**. One lucky correct answer does not prove mastery.
- It knows people **slip**. One careless mistake does not erase what they know.
- It tracks **confidence** separately. Being 80% sure after 3 questions is not the same as being 80% sure after 30, and the program treats those differently.

Say:

"Mastery is a Bayesian estimate, not a running total. It models guessing and slipping, and it gives us a confidence number the routing logic actually uses."

2.3 "The model proposes, the guard disposes"

What it is. This is our architecture in five words, and the thing the team is most proud of. An AI model suggests what to do next - retry, move on, go back a step. But it does not get the final say. A separate piece of ordinary, predictable code called **the guard** checks the suggestion against fixed rules, and overrules it when it breaks one.

Example of a rule the guard enforces: *you may not tell a student to move on to the next topic if their mastery is below 0.6*. The AI might suggest it. The guard says no.

Why it matters - three reasons, and this is worth learning properly:

Reason	What it buys us
Safety	The AI cannot wreck someone's learning path, even if it says something strange.
Measurement	We can count how often the guard overrules the AI. That is a real number in our results.

Demo reliability	Because the routing is predictable code, the demo takes the same path every time - even if the AI phrases this
------------------	--

Say:

"The model proposes and the guard disposes. An LLM suggests the next action, and deterministic code validates it against hard rules and overrides it when it is invalid - and we log every override."

2.4 The safety invariant (our failure never costs the student)

What it is. Two kinds of bad thing can happen when a student submits code:

Their problem	Our problem
Their code has a syntax error	Our sandbox crashed
Their code crashes at runtime	The AI provider timed out
Their answer is wrong	Our database failed
Their code loops forever	The AI returned nonsense

The left column says something about the student, so it changes their mastery score. The right column says something about **us**, so it must change nothing.

The clever part: we did not do this with an `if` statement that somebody has to remember to write. The two kinds are different *types* in the code, and the function that updates mastery only accepts the first kind. If an infrastructure failure ever reached it, the program would refuse outright. It is enforced by the language, not by discipline.

Say:

"Student outcomes and system faults are disjoint types, and mastery is only reachable from the first. If our infrastructure breaks, the student does not pay for it - and that is enforced by the type system, not by a conditional someone has to notice in review."

2.5 RAG - teaching from real material

What it is. RAG stands for Retrieval-Augmented Generation. In plain terms: before the AI writes an exercise, we **search our own course notes** for the relevant section and hand it to the AI as source material.

Why it matters. Without it, the AI invents an exercise from whatever it happens to remember. With it, the exercise is grounded in the actual curriculum, and we can show the student which page it came from.

On screen you will see a line like `05_recursion.md#5.1 The idea` - that is the exact section the exercise was built from.

One detail worth knowing: we do not call RAG on every step. It runs at one point out of twelve, where it actually helps. If a judge asks why, the answer is that retrieval costs time and adds nothing to arithmetic.

Say:

"Retrieval is filtered by skill first, then searched semantically, and every generated problem records which section it was grounded in."

3. Reading the screen

During the demo, lines scroll past. Each one is a real decision the program made - not a printed message we wrote in advance. Here is how to read them.

Line you will see	What it means in plain English
[diagnose]	Working out which skill is weakest and what it depends on
[plan_action]	Choosing the topic, difficulty and teaching style for the next task
[retrieve]	Searching our course notes for the right section
[generate_problem]	The AI writing the exercise, grounded in what was retrieved
[execute]	Running the student's code in a sandbox
[misconception]	Working out WHY the answer was wrong - the important one
[mastery]	Updating our estimate of what the student knows
[guard_override]	The guard rejecting the AI's suggestion - point at this one
[adapt]	Deciding what to do next
[prereq_redirect]	THE MOMENT. Changing its own goal to a prerequisite
[recover]	An infrastructure failure being absorbed safely
[prereq_return]	Coming back to the original topic
[finalize]	Session finished

The three lines that matter most

If the recording is going fast, these are the three to slow down on:

- **[misconception]** - it says *why* they failed, not just that they failed. This is the evidence that CogniFlow diagnoses causes.
- **[prereq_redirect]** - the agent changing its own objective. This is the project.
- **[recover] ... mastery_untouched=True** - proof the safety invariant is real and not just a claim on a slide.

4. Your job in the video

The script splits the narration between two people. You are **Speaker B**.

	Speaker A (partner)	Speaker B (you)
Role	Narrates the architecture and the why	Drives the machine and reads what appears
Needs	The concepts in section 2	The run sheet in section 6
Talks about	The prerequisite redirect, the guard, the results	What is on screen right now, the sandbox, the recovery

Why you have the better half

Speaker B's lines are all descriptions of things visible on screen at the moment you say them. You are never asked to explain something abstract from memory - you are reading reality out loud. That is much easier to do well, and much harder for a judge to doubt.

The golden rule of this recording

Never say a thing the screen is not showing. Judges have watched a lot of narrated slideshows. Every claim we make is visible as real output. If you are ever unsure what to say, describe what is literally on the screen - that is always safe and always true.

5. Setup - recording day

Do these in order. Do not skip the first one.

Step 1 - Prove it works, before you record anything

```
cd D:\Downloads\BloodCoded
python run.py check      # one real call per AI provider
python run.py live       # the full demo against real AI models
```

You want to see **ALL CHECKS PASSED** and the line Learning path : recursion -> functions -> recursion.

Step 2 - Run it three times

Run python run.py live three times and check the learning path is **identical** every time. It should be - the routing is deterministic by design. If it varies, stop and tell your partner before recording. A demo you cannot reproduce is a demo that will fail in front of judges.

Step 3 - Screen setup

- Terminal at about 110 characters wide, font size large enough to read on a phone.
- Close every other window, notification, and browser tab.
- Dark terminal theme reads better on video than light.
- Check your microphone by recording ten seconds and playing it back.

Step 4 - Know your fallbacks

If this happens	Do this
A provider is rate-limited	Nothing. It fails over to the second provider automatically. Mention it - it is a feature.
The internet drops	Say so, run 'python run.py verify' which needs no internet, and continue.
Something looks wrong	Say what you actually see. Do not narrate over it.

6. The run sheet

Target length 4 minutes 30. The full word-for-word script is in docs/DEMO_SCRIPT.md - this is your operator's view of it: what you do, and roughly what you say.

Time	You do	You say (roughly)
0:00	Nothing - partner opens	-
0:45	Run 'python run.py live'	"Real diagnosis, real retrieval with citations, real code execution in a sandbox."
1:00	Let attempt 1 scroll. Point at [mastery]	"The student's code raised an error - mastery drops, and it retries a variation. So
1:30	STOP SCROLLING at [prereq_redirect]. Partner starts A's moment)	"We inject a sandbox failure. Watch the mastery number - it does not move. If our
2:15	Point at [execute] sandbox_error, then [return]	
2:45	Point at [prereq_return] and the final numbers	
3:15	Point at the seven [PASS] lines	"The demo verifies its own claims, and CI asserts the same path against the ever
3:45	Run 'python run.py ablation'	(A reads the numbers)
4:20	Nothing - partner closes	-

The single most important instruction

At **1:30**, when [prereq_redirect] appears, **stop scrolling and leave it on screen**. That line is the entire project. Give it room to breathe while your partner explains it. The most common way to ruin this video is to scroll past the best moment in it.

7. If something breaks while recording

It probably will not. But if it does, this is the correct response.

The rule

Say what happened, plainly, and keep going. Do not pretend it did not happen. Do not talk over it. A recovered failure demonstrates the recovery path we are claiming - it is genuinely better evidence than a run where nothing went wrong. The rulebook prohibits fabricating results; it does not prohibit things going wrong.

Problem	What it means	What to say
degraded=True on a line	An AI provider failed, so it used a deterministic template instead.	"The provider is instead using a deterministic template."
A long pause	It is waiting for the AI to reply	"That is a live API call - this is not a recording."
An error you do not recognize	Unknown	"Something went wrong there - let me re-run it." Then run 'python run.py'

8. Questions a judge may ask you

These are the ones aimed at Speaker B. Short, honest answers are better than long ones. **'I do not know, but I can show you where it is in the code' is an acceptable answer** and far better than guessing.

Question	Your answer
Is the redirect hardcoded?	No. It is graph traversal over live mastery estimates. There is a test that asserts the state transition
Is the sandbox real?	Yes - a separate process with a timeout and a minimal environment. Student code cannot read o
What happens if the AI is down?	It fails over to a second provider. If all providers fail, it uses a deterministic template and marks th
Did you write this yourselves?	Yes. Two contributors, both human, and the contributor list is checked automatically before every
What does not work yet?	There is no login, no dashboard and no diagnostic quiz - a new student starts from a seeded prof

9. Glossary

Term	Plain meaning
Agent	A program that decides its own next step, instead of following a fixed script.
LangGraph	The library we use to define those steps and the paths between them.
Node	One step in that structure - 'diagnose', 'grade', 'adapt'.
State	Everything the program currently knows about this session.
Checkpoint	A saved copy of that state, written to disk, so it can resume later.
Interrupt	The program genuinely stopping and waiting for the student - not a loop that spins.
Mastery	Our 0-to-1 estimate of whether the student knows a skill.
Confidence	How much evidence that estimate rests on.
BKT	Bayesian Knowledge Tracing - the method behind those two numbers.
Prerequisite	A skill you must know before another makes sense.
The guard	Predictable code that checks and can overrule the AI's suggestion.
RAG	Searching our own notes and giving them to the AI as source material.
Sandbox	The isolated place student code runs, so it cannot harm anything.
SystemFault	A failure that is ours, not the student's. Never changes mastery.
Ablation	An experiment where you remove one part to prove it was doing something.

10. What to learn before 9 September

You do not need all of this. In priority order - if you only do the first row, you can still record a good video.

Priority	What	Time	How
Essential	Section 1 of this guide, until you can say the 30-line idea without a hint	30 min	Read it, then explain it out loud to someone
Essential	Watch the demo run three times and follow the lines in <code>section3un.py</code> live, guide open beside you	45 min	
Essential	Rehearse the run sheet twice with your partner	1 hour	Time it. Aim for 4:30.
Strongly	Sections 2.3 and 2.4 - the guard and the safety invariant	45 min	These are the two a judge is most likely to probe
Strongly	The Q&A table in section 8	30 min	Say the answers aloud, do not just read
If time	Section 2.2 - what BKT actually does	45 min	Optional. The one-sentence version is enough.
If time	Open <code>app/mastery/policy.py</code> and read the rules	1 hour	It is ordinary Python with comments. Less scary than it sounds

A final thought

The strongest thing you can do in a technical Q&A; is be straight about the limits. We know what CogniFlow does not do yet - there is no login, no dashboard, no diagnostic quiz - and saying so plainly makes every other claim more believable, not less. Confidence is knowing where the edges are, not pretending there are none.

Good luck. The engine works - all you have to do is show it honestly.